

Diviser pour régner

<https://www.developpement-informatique.com>

Mr. ESSADDOUKI Mostafa

Professeur agrégé en informatique

En informatique, la résolution de problèmes complexes nécessite souvent des méthodologies systématiques pour concevoir des algorithmes efficaces.

Plutôt que de traiter un problème de manière brute (force brute), on adopte des paradigmes de conception qui fournissent des cadres généraux.

Les plus importants sont :

- ❑ Diviser pour régner
- ❑ Approches gloutonnes
- ❑ Programmation dynamique
- ❑ Heuristiques et métaheuristiques

Diviser pour régner

Diviser pour régner est une stratégie algorithmique récursive qui consiste à décomposer un problème complexe en plusieurs sous-problèmes plus simples.

Cette stratégie repose sur trois étapes :

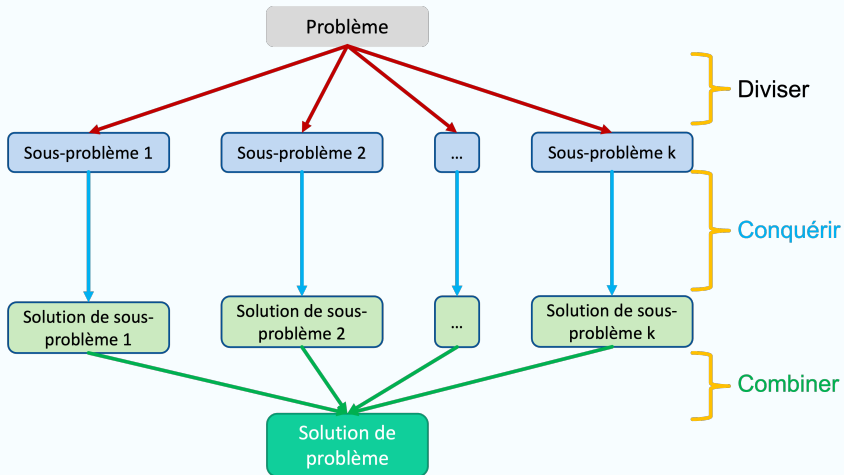
- 1 **Diviser** : scinder le problème en un ou plusieurs sous-problèmes de taille plus réduite.
- 2 **Conquérir** : résoudre chaque sous-problème, généralement de manière récursive, jusqu'à atteindre des cas suffisamment simples pour être traités directement.
- 3 **Combiner** : rassembler les solutions des sous-problèmes afin de reconstruire la solution complète du problème de départ.

■ Méthode générale:

On peut formaliser la stratégie de Diviser pour régner par une fonction récursive :

```
Fonction DiviserRegner(Problème P)
  SI (P est assez simple ou de petite taille) ALORS
    Résoudre directement P
  SINON
    Diviser P en sous-problèmes P1, P2, ..., Pk
    Pour chaque sous-problème Pi
      Solution_i ← DiviserRegner(Pi)
    FinPour
    Retourner Combiner(Solution_1, Solution_2, ... , Solution_k)
  FINSI
FINFonction
```

Diviser pour régner



■ Comment mettre en place cette stratégie ?:

La conception d'un algorithme basé sur cette stratégie repose sur quatre étapes essentielles :

- 1 Choisir la manière de diviser le problème
- 2 Déterminer les paramètres d'entrée des appels récursifs
- 3 Identifier le cas de base (ou cas trivial)
- 4 Définir l'opération de combinaison

☞ Choisir la manière de diviser le problème:

Identifier les bons sous-problèmes qui, une fois résolus, permettront de reconstruire la solution complète.

👉 Identifier le cas de base (ou cas trivial):

Il est indispensable de définir la version la plus simple du problème, dont la solution est immédiate.

Ce cas de base sert de condition d'arrêt pour la récursion et empêche l'algorithme de tourner indéfiniment.

👉 *Exemple(s) 1:*

Dans la recherche dichotomique, si le tableau ne contient qu'un seul élément, il suffit de vérifier si c'est la valeur cherchée.

👉 Définir l'opération de combinaison:

Une fois que chaque sous-problème est résolu, il faut combiner leurs résultats pour obtenir la solution du problème initial.

La nature de cette opération dépend du problème :

- ➡ Concaténation ou fusion pour les algorithmes de tri.
- ➡ Addition ou agrégation pour les calculs numériques.
- ➡ Comparaison ou sélection pour la recherche d'optimum.

Problème : Étant donné un tableau trié T de taille n et une valeur x , déterminer si x est présent dans T .

👉 Etape 1 – Diviser:

n compare x avec l'élément du **milieu** du tableau : $T[m]$ avec $m = \frac{(gauche+droite)}{2}$.

Deux sous-problèmes potentiels apparaissent :

- ➡ Si $x < T[m]$, alors la recherche est réduite à la **moitié gauche** du tableau.
- ➡ Si $x > T[m]$, alors la recherche est réduite à la **moitié droite** du tableau.

Ainsi, le problème initial (taille n) est divisé en un problème plus petit (taille $\frac{n}{2}$).

👉 Etape 2 – Déterminer les paramètres d'entrée:

Chaque sous-problème est défini par les **bornes gauche et droite** du sous-tableau considéré.

Appel récursif : '**RechercheDicho(T, gauche, droite, x)**'

➡ Cas gauche : '**RechercheDicho(T, gauche, m-1, x)**'

➡ Cas droit : '**RechercheDicho(T, m+1, droite, x)**'

👉 Etape 3 – Identifier le cas de base:

- ❑ Si ' $x == T[m]$ ', on a trouvé la valeur : **succès**.
- ❑ Si ' $\text{gauche} > \text{droite}$ ', cela signifie que le tableau est vide $\rightarrow$ **échec** (l'élément n'existe pas).

Ces deux cas représentent les versions **triviales** du problème.

👉 Etape 4 – Combiner:

Ici, la **combinaison** est simple : il n'y a pas de fusion compliquée à faire.

On retourne directement le résultat de la recherche dans la sous-partie choisie (gauche ou droite).

La solution finale est donc le **résultat renvoyé par l'unique sous-problème pertinent**.

- ❑ Le **tri fusion** est un algorithme de tri fondé directement sur la stratégie *Diviser pour régner*.
- ❑ Il repose sur l'idée de découper un tableau en parties plus petites, de trier chacune d'elles récursivement, puis de fusionner les résultats pour obtenir un tableau trié.

👉 Diviser:

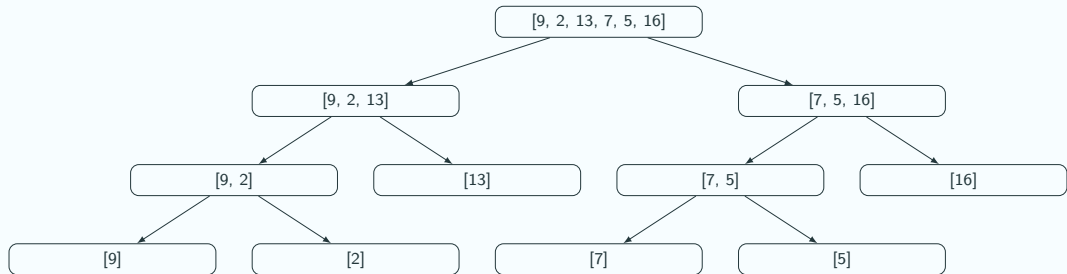
On coupe le tableau en deux parties de taille à peu près égale :

- ❑ Sous-tableau gauche
- ❑ Sous-tableau droit

👉 Conquérir (résolution récursive):

- ❑ Chaque sous-tableau est trié récursivement à l'aide de la même méthode Tri Fusion.
- ❑ Lorsque les sous-tableaux atteignent une taille de 1 (ou 0), ils sont considérés comme déjà triés (cas de base).

👉 **Diviser:**

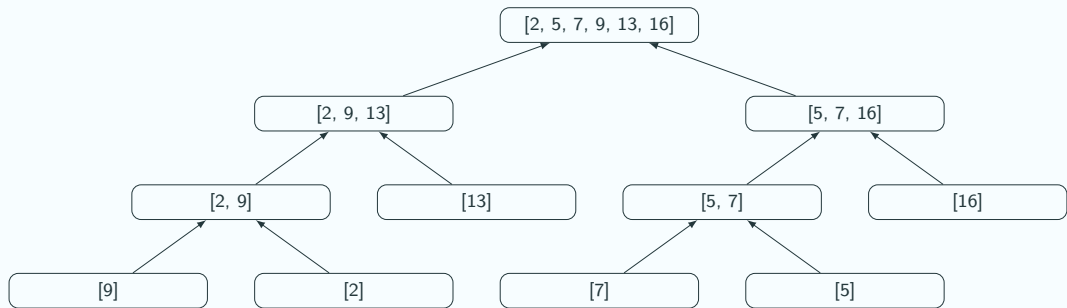


👉 Combiner (fusionner):

On fusionne les deux sous-tableaux triés pour former un tableau unique, également trié.

La fusion s'effectue en comparant élément par élément, en insérant toujours le plus petit des deux premiers éléments restants.

Tri fusion



■ Pseudo-code du tri fusion:

```
Fonction TriFusion(T)
  SI taille(T) <= 1 ALORS
    retourner T
  SINON
    m ← taille(T) / 2
    gauche ← TriFusion(T[0..m-1])
    droite ← TriFusion(T[m..n-1])
    retourner Fusion(gauche, droite)
  FINSI
FINFonction
```

■ Principe de l'algorithme de fusion:

On suppose que l'on dispose de deux sous-tableaux déjà triés :

⇒ G (gauche)

⇒ D (droit)

L'objectif est de produire un tableau résultat contenant tous les éléments de G et D, en ordre croissant.

■ Principe de l'algorithme de fusion:

1 Initialisation :

- ❑ On crée un tableau vide **Résultat**.
- ❑ On place deux pointeurs i et j au début des deux sous-tableaux G et D .

2 Comparaison des éléments courants

- ❑ On compare $G[i]$ et $D[j]$.
- ❑ Le plus petit des deux est ajouté à **Résultat**.
- ❑ On avance le pointeur correspondant (i ou j).

3 Répétition : On continue à comparer les éléments tant que les deux sous-tableaux contiennent encore des éléments non traités.

4 Ajout des éléments restants : Quand l'un des sous-tableaux est vide, on ajoute tous les éléments restants de l'autre sous-tableau à **Résultat**.

■ Principe de l'algorithme de fusion:

```
Fonction Fusion(G, D)
    resultat ← tableau vide
    Tant que G et D ne sont pas vides
        SI G[i] <= D[j] ALORS
            ajouter G[i] à resultat et supprimer G[i] de G
            i ← i + 1
        SINON
            ajouter D[j] à resultat et supprimer D[j] de D
            j ← j + 1
    FinTantQue
    Ajouter les éléments restants de G ('sil y en a) à resultat
    Ajouter les éléments restants de D ('sil y en a) à resultat
    retourner resultat
FINFonction
```

Le **tri rapide** est un algorithme de tri qui utilise également la stratégie *Diviser pour régner*, mais avec une différence importante par rapport au tri fusion : la division se fait à l'aide d'un pivot, et il n'y a pas d'étape explicite de fusion.

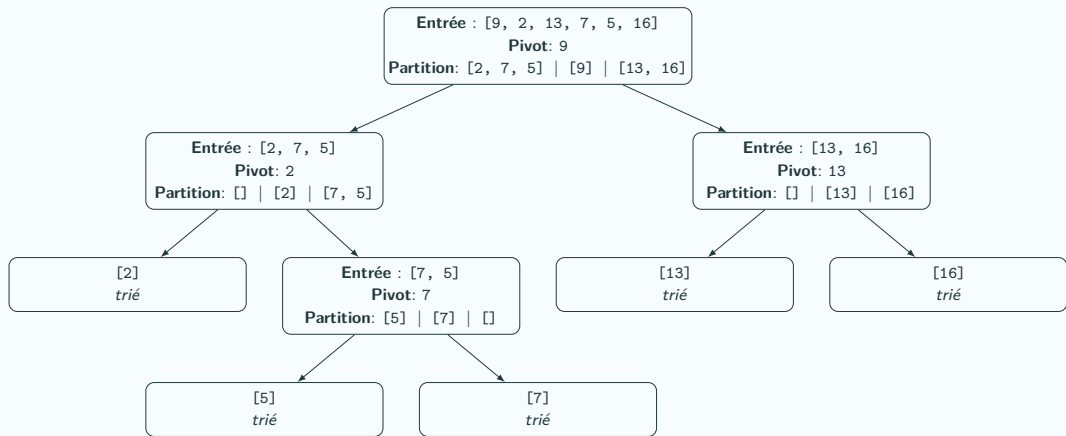
👉 Diviser (partitionner):

- ❑ On choisit un élément du tableau appelé pivot.
- ❑ On réarrange les autres éléments du tableau de sorte que :
 - ➡ tous les éléments plus petits que le pivot soient placés avant lui,
 - ➡ tous les éléments plus grands que le pivot soient placés après lui.
- ❑ Après cette opération, le pivot est à sa position finale dans le tableau trié.

☞ Conquérir (appel récursif):

On applique récursivement le tri rapide sur la partie gauche (**éléments** < **pivot**) et sur la partie droite (**éléments** > **pivot**).

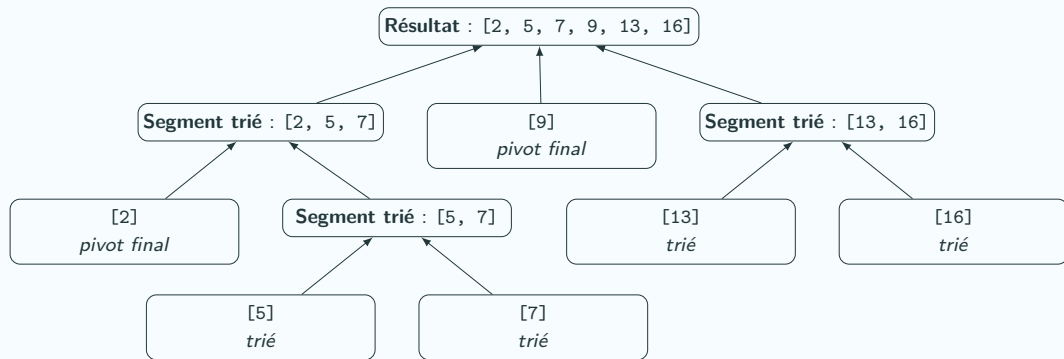
Tri rapide



👉 Combiner:

Contrairement au tri fusion, aucune fusion explicite n'est nécessaire : le pivot joue le rôle de séparation, donc la combinaison est **implicite**.

Tri rapide



■ Pseudo-code du tri rapide:

```
Fonction TriRapide(T, gauche, droite)
    SI gauche < droite ALORS
        p ← Partition(T, gauche, droite)
        TriRapide(T, gauche, p-1)
        TriRapide(T, p+1, droite)
    FINSI
FINFonction
```

■ Pseudo-code de partitionnement:

```
Fonction Partition(T, gauche, droite)
    pivot ← T[droite]
    i ← gauche - 1
    Pour j de gauche à droite - 1
        SI T[j] ≤ pivot ALORS
            i ← i + 1
            échanger T[i] et T[j]
    FinPour
    échanger T[i+1] et T[droite]
    retourner i+1 // position finale du pivot
FINFonction
```

La **complexité temporelle** d'un algorithme récursif se traduit souvent par une **équation de récurrence**, qu'il faut résoudre pour obtenir un ordre de grandeur asymptotique.

La **complexité temporelle** d'un algorithme récursif se traduit souvent par une **équation de récurrence**, qu'il faut résoudre pour obtenir un ordre de grandeur asymptotique.

Une **récurrence** exprime le temps d'exécution $T(n)$ d'un problème de taille n en fonction du temps d'exécution de sous-problèmes plus petits.

La **complexité temporelle** d'un algorithme récursif se traduit souvent par une **équation de récurrence**, qu'il faut résoudre pour obtenir un ordre de grandeur asymptotique.

Une **récurrence** exprime le temps d'exécution $T(n)$ d'un problème de taille n en fonction du temps d'exécution de sous-problèmes plus petits.

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

- ❑ a = nombre de sous-problèmes générés
- ❑ n/b = taille de chaque sous-problème
- ❑ $f(n)$ = coût du travail supplémentaire (division, combinaison, etc.)

Méthode de substitution

Dans cette méthode on remplace l'expression récursive par sa définition plusieurs fois de suite pour révéler un motif sous-jacent.

Une fois que ce motif est identifié, on peut en déduire une forme fermée, ce qui nous permet de calculer la complexité. Étapes pour dérouler une récurrence :

- 1 Formuler la récurrence de base
- 2 Substituer plusieurs fois
- 3 Généraliser le motif
- 4 Trouver la valeur de k pour atteindre le cas de base
- 5 Substituer la valeur de k et simplifier
- 6 Conclure sur la complexité

☞ *Exemple(s) 1:*

$$T(n) = T(n - 1) + c$$

👉 Exemple(s) 2:

$$T(n) = T(n - 1) + c$$

👉 **Substituer plusieurs fois:**

$$\begin{aligned} T(n) &= T(n - 1) + c \\ &= [T(n - 2) + c] + c = T(n - 2) + 2c \\ &= [T(n - 3) + c] + 2c = T(n - 3) + 3c \end{aligned}$$

👉 Généraliser le motif:

On observe la séquence et on trouve une formule générale pour le k-ième remplacement. Dans notre exemple, après k étapes, le motif semble être

$$T(n) = T(n - k) + kc$$

.

👉 Généraliser le motif:

On observe la séquence et on trouve une formule générale pour le k -ième remplacement. Dans notre exemple, après k étapes, le motif semble être

$$T(n) = T(n - k) + kc$$

.

👉 Trouver la valeur de k pour atteindre le cas de base:

On sait que la récurrence s'arrête à un cas de base, souvent $T(1)$ ou $T(0)$. On résout l'équation pour trouver la valeur de k qui nous mène au cas de base. Si le cas de base est $T(1)$, on pose $n - k = 1$, ce qui donne $k = n - 1$.

☞ Substituer la valeur de k et simplifier:

On remplace k dans la formule généralisée.

$$\begin{aligned}T(n) &= T(n - k) + kc \\&= T(n - (n - 1)) + (n - 1)c \\&= T(1) + (n - 1)c\end{aligned}$$

☞ Conclure sur la complexité:

Le résultat final montre la complexité en fonction de n . Dans notre exemple, si on suppose que $T(1)$ est une constante (par exemple 1), alors

$$T(n) = 1 + (n - 1)c = 1 + cn - c$$

. C'est une fonction linéaire en n . On peut donc conclure que la complexité est

$$T(n) = O(n)$$

.

👉 *Exemple(s) 3:*

$$T(n) = 2T\left(\frac{n}{2}\right) + n.$$

👉 **Substituer plusieurs fois:**

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 2[2T(n/4) + n/2] + n &= 4T(n/4) + 2n \\ &= 4[2T(n/8) + n/4] + 2n = 8T(n/8) + n + 2n = 8T(n/8) + 3n \end{aligned}$$

👉 Généraliser le motif:

On peut voir un motif après k étapes :

- ❑ Le coefficient devant T est 2^k
- ❑ Le terme à l'intérieur de T est $n/2^k$
- ❑ Le terme ajouté est kn .

Donc,

$$T(n) = 2^k T(n/2^k) + kn$$

👉 Généraliser le motif:

On peut voir un motif après k étapes :

- ❑ Le coefficient devant T est 2^k
- ❑ Le terme à l'intérieur de T est $n/2^k$
- ❑ Le terme ajouté est kn .

Donc,

$$T(n) = 2^k T(n/2^k) + kn$$

👉 Trouver la valeur de k :

Le cas de base est $T(1)$. On pose $n/2^k = 1$, ce qui implique $n = 2^k$, soit $k = \log_2(n)$

☞ Substituer la valeur de k et simplifier:

$$T(n) == 2^k T(n/2^k) + kn = nT(1) + n(\log_2 n)$$

☞ **Substituer la valeur de k et simplifier:**

$$T(n) == 2^k T(n/2^k) + kn = nT(1) + n(\log_2 n)$$

☞ **Conclure sur la complexité:**

Si $T(1)$ est une constante, alors $T(n) = n.C + n\log_2 n$.

Puisque le terme $n\log n$ domine le terme n pour les grandes valeurs de n , la complexité est $O(n\log n)$.

Théorème maître

Le théorème maître s'applique aux récurrences de la forme :

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

où :

- ❑ $a \geq 1$: nombre de sous-problèmes générés,
- ❑ $b > 1$: facteur de réduction de la taille du problème,
- ❑ $f(n)$: coût du travail hors récursion (division + combinaison).

☞ **Cas 1 : Le coût de la division/combinaison est dominé par les sous-problèmes.:**

Si $f(n) = O(n^{\log_b a - \epsilon})$ pour une constante $\epsilon > 0$. Autrement dit, si $f(n)$ est asymptotiquement plus petite que $n^{\log_b a}$. Alors, la solution est $T(n) = O(n^{\log_b a})$.

☞ **Cas 1 : Le coût de la division/combinaison est dominé par les sous-problèmes.:**

Si $f(n) = O(n^{\log_b a - \epsilon})$ pour une constante $\epsilon > 0$. Autrement dit, si $f(n)$ est asymptotiquement plus petite que $n^{\log_b a}$. Alors, la solution est $T(n) = O(n^{\log_b a})$.

☞ **Cas 2 : Le coût est réparti uniformément.:**

Si $f(n) = O(n^{\log_b a})$. Alors, la solution est $T(n) = O(n^{\log_b a} \log n)$.

☞ Cas 3 : Le coût de la division/combinaison domine.:

Si $f(n) = \Omega(n^{\log_b a + \epsilon})$ pour une constante $\epsilon > 0$. Autrement dit, si $f(n)$ est asymptotiquement plus grande que $n^{\log_b a}$. De plus, si $a \cdot f(n/b) \leq c \cdot f(n)$ pour une constante $c < 1$ et pour tout n assez grand (cette condition est la **condition de régularité**). Alors, la solution est $T(n) = O(f(n))$.

👉 Exemple(s) 4: Cas 1

$$T(n) = 8T\left(\frac{n}{2}\right) + n^2$$

- ❑ $a = 8, b = 2, f(n) = n^2$.
- ❑ On calcule $n^{\log_b a} = n^{\log_2 8} = n^3$.
- ❑ On compare $f(n) = n^2$ à n^3 . Puisque $n^2 = O(n^{3-\epsilon})$ (on peut choisir $\epsilon = 1$), on est dans le Cas 1.
- ❑ La solution est $T(n) = O(n^3)$.

👉 Exemple(s) 5: Cas 2

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

- ❑ $a = 2, b = 2, f(n) = n$.
- ❑ On calcule $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.
- ❑ On compare $f(n) = n$ à n^1 . Puisque $n = O(n^1)$, on est dans le Cas 2.
- ❑ La solution est $T(n) = O(n \log n)$.

👉 Exemple(s) 6: Cas 2

$$T(n) = 3T\left(\frac{n}{4}\right) + n \log n$$

- ❑ $a = 3$, $b = 4$, $f(n) = n \log n$.
- ❑ On calcule $n^{\log_b a} = n^{\log_4 3} \approx n^{0,79}$.
- ❑ On compare $f(n) = n \log n$ à $n^{0,79}$. Puisque $n \log n$ est asymptotiquement plus grand, on a $f(n) = \Omega(n^{\log_4 3 + \epsilon})$ (on peut choisir $\epsilon = 0,21$).
- ❑ On vérifie la condition de régularité :
 $a \cdot f(n/b) = 3 \cdot (n/4) \log(n/4) = (3/4)n(\log n - \log 4)$.
- ❑ On veut $a \cdot f(n/b) \leq c \cdot f(n)$, soit $(3/4)n(\log n - \log 4) \leq c \cdot n \log n$. On peut choisir $c = 3/4 < 1$ pour n assez grand. La condition est vérifiée.
- ❑ La solution est $T(n) = O(f(n)) = O(n \log n)$.

■ Limites du Théorème Maître:

Le théorème maître ne s'applique pas si :

- ❑ $f(n)$ n'est pas asymptotiquement comparable à $n^{\log_b a}$. Par exemple, $f(n) = n^{\log_b a} / \log n$.
- ❑ La condition de régularité du Cas 3 n'est pas remplie.
- ❑ a n'est pas une constante ($a < 1$).
- ❑ n/b n'est pas une division de n par une constante.

Tri Fusion : Analyse de la complexité

L'algorithme **Tri Fusion** fait deux opérations principales :

- 1 **Diviser le tableau en deux sous-tableaux** : coût $\approx O(1)$ (simple calcul de l'indice du milieu).
- 2 **Appeler récursivement TriFusion** sur les deux sous-tableaux $\rightarrow$ deux appels sur des sous-problèmes de taille $\frac{n}{2}$.
- 3 **Fusionner les deux sous-tableaux triés** : coût proportionnel à la taille du tableau $\rightarrow O(n)$.

Donc, la relation de récurrence est :

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

où **c** est une constante (provenant de la fusion et des comparaisons).

■ Résolution de la récurrence:

❑ Niveau 1 : $T(n) = 2T(n/2) + cn$

❑ Niveau 2 : $T(n/2) = 2T(n/4) + c(n/2) \rightarrow$ donc :
$$T(n) = 2[2T(n/4) + c(n/2)] + cn = 4T(n/4) + 2cn$$

❑ Niveau 3 : $T(n) = 8T(n/8) + 3cn$

❑ ...

❑ Niveau k : $T(n) = 2^k T(n/2^k) + kcn$

■ Résolution de la récurrence:

👉 Condition d'arrêt:

Quand $n/2^k = 1 \rightarrow k = \log_2 n$. Donc à la profondeur maximale :

$$T(n) = nT(1) + cn \log_2 n$$

Or, $T(1)$ est une constante (coût de résoudre un tableau de taille 1). Donc :

$$T(n) = O(n) + O(n \log n) = O(n \log n)$$

Tri rapide : Analyse de la complexité

L'algorithme **Tri rapide** fait deux opérations principales :

- 1 Étape Partition : coûte $O(n)$ (parcours du tableau une fois).
- 2 Deux appels récurifs : tailles dépendantes du pivot.

Donc, la récurrence est :

$$T(n) = T(k) + T(n - k - 1) + O(n)$$

où 'k' est la taille de la sous-partie gauche (le pivot consomme 1 élément).

Tri rapide : Analyse de la complexité

Si le pivot coupe le tableau en deux parties de tailles approximativement égales :

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

C'est la même équation que le **tri fusion** → solution : $O(n \log n)$.

Si le pivot est toujours le plus petit ou le plus grand élément (ex. tableau déjà trié sans pivot aléatoire) :

$$T(n) = T(n-1) + O(n)$$

■ La résolution de la récurrence:

$$\begin{aligned}T(n) &= T(n-1) + cn \\&= [T(n-2) + c(n-1)] + cn \\&= T(n-2) + c(n-1) + cn \\&= [T(n-3) + c(n-2)] + c(n-1) + cn \\&= T(n-3) + c(n-2) + c(n-1) + cn \\&\dots \\&= T(1) + c(2 + 3 + \dots + n)\end{aligned}$$

■ La résolution de la récurrence:

La somme $2 + 3 + \dots + n$ est équivalente à :

$$\frac{n(n+1)}{2} - 1$$

Donc :

$$T(n) = T(1) + c \left(\frac{n(n+1)}{2} - 1 \right)$$

$$T(n) = O(n^2)$$